

Table of Contents

Question 1.....	2
(a) Third Type Calculator.....	2
(b) Program Segment.....	8
Question 2.....	15
(a) Identify the lexical specification.....	15
(b) Finite State Diagram.....	20
(c) Lexical analyzer.....	25
Part 1 - Using Flex and verified token is correct	25
Part 2 – Using both flex and bison verification.	28
(d) Lexical Error recognized.	33
(e) Test Cases	35
Part 1 – flex only	35
Part 2 – both flex and bison	38

Question 1

Download the attachments 'LexAndYacc.pdf' and 'LexAndYaccCode.zip' from the Moodle page (or refer at <http://epaperpress.com/lexandyacc/index.html>) and study the files in the attachments to answer the following:

(a) Summarize the procedures that the author has followed for the third type of calculator.

(a) Third Type Calculator

For the third type calculator Tom Niemann use six code files and below I summarized the procedures that use in each code files.

```
F:\NAWAM\Calculator>bison -y -d calc3.y
F:\NAWAM\Calculator>flex calc3.l
F:\NAWAM\Calculator>gcc -c y.tab.c lex.yy.c
F:\NAWAM\Calculator>gcc y.tab.o lex.yy.o calc3a.c -o calc3a.exe
F:\NAWAM\Calculator>gcc y.tab.o lex.yy.o calc3b.c -o calc3b.exe
F:\NAWAM\Calculator>gcc y.tab.o lex.yy.o calc3g.c -o calc3g.exe
F:\NAWAM\Calculator>
```

```
F:\NAWAM\Calculator>calc3a.exe
```

```
x = 0;
while (x < 3) {
    print x;
    x = x + 1;
}
0
1
2
^Z
```

```
F:\NAWAM\Calculator>calc3b.exe
```

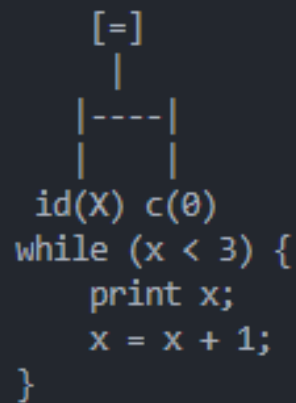
```
x = 0;
        push    0
        pop     x
while (x < 3) {
    print x;
    x = x + 1;
}
L000:
        push    x
        push    3
        compLT
        jz      L001
        push    x
        print
        push    x
        push    1
        add
        pop     x
        jmp     L000
L001:
^Z
```

```
F:\NAWAM\Calculator>
```

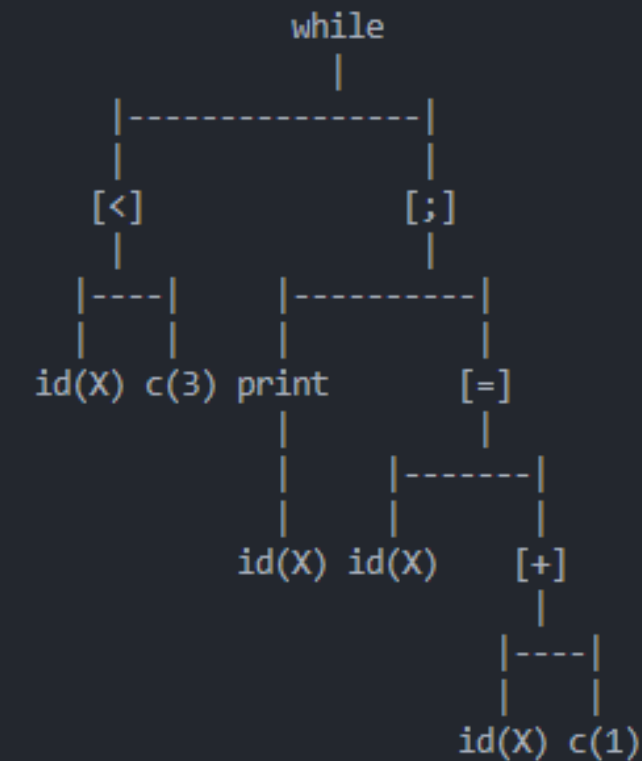
```
F:\NAWAM\Calculator>calc3g.exe
```

```
x = 0;
```

Graph 0:



Graph 1:



^Z

```
F:\NAWAM\Calculator>
```

1. Include File - calc3.h

- The structures for the syntax tree and symbol table.
- It includes an **enum (nodeEnum)** to distinguish node types (constant, identifier, operator).
- Structures (**conNodeType**, **idNodeType**, **oprNodeType**) are defined for constants, identifiers, and operators respectively.
- A **union (nodeType)** encapsulates these structures with a type field to identify the node type.
- A symbol table array (**sym**) is used to store single-character variable names.

2. Lex Input File - calc3.l

- The Lex file defines rules for tokenizing the input.
- Patterns are specified for matching variables (**[a-z]**), integers (**0** and **[1-9][0-9]***), single-character operators (**[-()<>=+*/;{}.]**), and multi-character operators (**">="**, **"<="**, **"=="**, **"!="**).
- Keywords of **"while"**, **"if"**, **"else"**, and **"print"** are also defined.
- Whitespace is ignored, and error handling is included for unknown characters.

3. Yacc Input File - calc3.y

- Its include the grammar and parsing rules and constructs the syntax tree.
- Tokens and operator precedence are specified to resolve ambiguities(if else ambiguity removing).
- Grammar rules are defined for various components of derivations.
 1. **program**- Starts parsing and exits after processing all statements.
 2. **function** - Processes and executes statements.
 3. **stmt** - Handles different types of statements, including expressions, print statements, variable assignments, while loops, and if-else statements.
 4. **Stmt_list** - Manages lists of statements.

5. **Expr** - Manages arithmetic operations and comparisons.

- Token and Non-Terminal Type Definitions

Directive	Purpose	Details
%union	Defines a union for YYSTYPE	Members • int iValue: Integer values • char sIndex: Symbol table index (variable names) • nodeType *nPtr: Node pointers in the abstract syntax tree
%token	Declares token types	Tokens • <iValue> INTEGER: Integer values • <sIndex> VARIABLE: Variable names • WHILE, IF, PRINT: Keywords in the language
%nonassoc	Specifies non-associative operators	Operators • IFX: Controls precedence and associativity of the IF statement
%left	Declares left-associative operators	Operators GE : Greater than or equal to, declared left associative.
%type	Declares the type of non-terminal symbols	Non-terminals • <nPtr> stmt: Statements • <nPtr> expr: Expressions • <nPtr> stmt_list: List of statements

4. Calculator Interpreter - **calc3a.c**

- The interpreter evaluates the syntax tree and executes operations.
- The function **ex(nodeType *p)** recursively evaluates the syntax tree nodes, handling constants, identifiers, operators, and control statements like **WHILE, IF, and PRINT**.

5. Calculator Compiler - **calc3b.c**

- The compiler generates code for a hypothetical stack-based machine from the syntax tree.

- The function **ex(nodeType *p)** generates machine instructions corresponding to the syntax tree nodes, including handling for loops, conditionals, arithmetic operations, and assignments.

6. Calculator Graph - calc3g.c

- This component generates a visual representation of the syntax tree.
- Graph drawing functions (**graphInit()**, **graphFinish()**, **graphBox()**, **graphDrawBox()**, **graphDrawArrow()**) are used to create and draw the graph.
- The function **exNode(nodeType *p, int c, int l, int *ce, int *cm)** recursively traverses the syntax tree and draws nodes and connections.

This calculator show case clearly about the phases in the compiler also.

Compiler Phase	File	Procedures
Lexical Analysis	calc3.l	• Token Recognition: Uses regular expressions to recognize tokens such as variables (single lowercase letters), integers (0-9), operators (+, -, *, /, etc.), and keywords (while, if, else, print). • Token Value Assignment: Assigns values to yylval for tokens, converting text to corresponding internal representations. • Error Handling: Reports unknown characters encountered during scanning.
Syntax Analysis	calc3.y	• Grammar Definition: Defines grammar rules for expressions, statements, and programs using Bison. • AST Node Construction: Uses helper functions (opr, id, con) to construct abstract syntax tree (AST) nodes. • Operator Precedence and Associativity: Specifies precedence and associativity for operators to resolve ambiguities in expressions.
Semantic Analysis	calc3.y	• Symbol Table Management: Manages a symbol table (sym) for variables. • Semantic Actions: Implements actions in grammar rules to build AST and perform semantic checks.
Intermediate Code Generation	calc3a.c	• AST Evaluation: The ex function recursively evaluates AST nodes to execute the program. • Execution of Constructs: Handles arithmetic operations, control structures (while loops, if-else conditions), and print statements.

Code Generation	calc3b.c	• Instruction Generation: The ex function generates stack-based instructions (push, add, sub, etc.) for a virtual machine. • Control Flow Translation: Translates high-level constructs into low-level control flow instructions (jmp, jz, etc.).
Graphical Representation	calc3g.c	• Syntax Tree Visualization: Functions to draw and visualize the syntax tree. • Graph Drawing Functions: Includes graphInit, graphFinish, graphBox, graphDrawBox, and graphDrawArrow to create graphical representations. • Recursive Tree Processing: The exNode function processes AST nodes recursively to generate a visual graph.
Memory Management	calc3.y	• Node Allocation and Deallocation: Functions (con, id, opr, freeNode) handle dynamic memory management for AST nodes to prevent memory leaks.

(b) Consider the following program segment:

```

p = 15 ; q = 3 ;
while ( p > q ) {
    if p < q then
        p = p - q ;
    else
        q = p + q ;
}
print q ;

```

Apply the steps you summarized in (a) to obtain the lex and yacc files for the above program segment and attach the both files with output results each.

(b) Program Segment

```

p = 15 ; q = 3 ;
while ( p > q ) {
    if p < q then
        p = p - q ;
    else
        q = p + q ;
}

```

```
print q;
```

- The above code segment first needs to clarify the lexical phase, according to that variable **p** and **q** which can be the same as the regular expression in the calculator.

Calc3.1 snippet code -

```
[a-z] {  
    yyval.sIndex = *yytext - 'a';  
    return VARIABLE;  
}
```

- Symbol of **"="**, **";"**, **"("**, **")"**, **"{"**, **"}"**, **"<"**, **">"**, **"="**, **"+"**, **"-"** are also accept same as use in the calculator.

Calc3.1 snippet code -

```
[-0<=>+*/;{}.] {  
    return *yytext;  
}
```

- When it comes to the key words **while**, **if**, **then**, **else**, **print** only one word cannot recognize and that word is **then**.

Calc3.1 snippet code -

```
"while"    return WHILE;  
"if"       return IF;  
"else"     return ELSE;  
"print"    return PRINT;
```

To run the above code segment first I need to declare the if as a token in the Lex input file called **calc3.1**. After that I need to parser it to the yacc file and declare as a token in the yacc file and under the rule of stmt I need to check how is it happening?.

```

stmt:
    ';'                                { $$ = opr(';', 2, NULL, NULL); }
  | expr ';'                          { $$ = $1; }
  | PRINT expr ';'                    { $$ = opr(PRINT, 1, $2); }
  | VARIABLE '=' expr ';'            { $$ = opr('=', 2, id($1), $3); }
  | WHILE '(' expr ')' stmt          { $$ = opr(WHILE, 2, $3, $5); }
  | IF '(' expr ')' stmt %prec IFX   { $$ = opr(IF, 2, $3, $5); }
  | IF '(' expr ')' stmt ELSE stmt   { $$ = opr(IF, 3, $3, $5, $7); }
  | '{' stmt_list '}'               { $$ = $2; }
;

```

In the abovementioned snippet code clearly state that if condition should cover inside the parentheses. I must change by removing the parentheses and add the **then** key word. And have to declare as token in the Yacc. I attached the both code and the output with the changes that run to the above segment code.

Summary of Changes

- **Lexer (calc3.l)** - Recognize the **THEN** keyword.
- **Parser (calc3.y)** - Update grammar rules to handle **IF expr THEN stmt** without parentheses.
- **Expression Execution (calc3a.c)** - Ensure correct interpretation of the new **IF expr THEN stmt** structure.
- **Code Generation (calc3b.c)** - Ensure code generation handles the new **IF expr THEN stmt** structure.
- **Graph Generation (calc3g.c)** - Ensure the visual representation reflects the new syntax.

By updating all relevant parts of the codebase, you ensure that the new **IF expr THEN stmt** syntax is correctly recognized, executed, and represented in the generated code and graphical output.

Changes to the calc3.l file

1. Added the keyword **then** to the lex rules.
2. Modified the comment to note that **then** is a new keyword.

```

">="      return GE;
"<="      return LE;
"=="      return EQ;
"!="      return NE;
"while"    return WHILE;
"if"       return IF;
"then"     return THEN; /* the new key word added */
"else"     return ELSE;
"print"    return PRINT;

```

Changes to the calc3.y file

1. Added the **THEN** token.
2. Updated the rules for **IF** to use **THEN** instead of parentheses for the if-then-else statements

```
%token <iValue> INTEGER
%token <sIndex> VARIABLE
%token WHILE IF THEN PRINT    /* THEN key word take as a token */
%nonassoc IFX
%nonassoc ELSE
```

```
stmt:
| ';' { $$ = opr(';', 2, NULL, NULL); }
| expr ';' { $$ = $1; }
| PRINT expr ';' { $$ = opr(PRINT, 1, $2); }
| VARIABLE '=' expr ';' { $$ = opr('=', 2, id($1), $3); }
| WHILE '(' expr ')' stmt { $$ = opr(WHILE, 2, $3, $5); }
// | IF '(' expr ')' stmt %prec IFX { $$ = opr(IF, 2, $3, $5); }
// | IF '(' expr ')' stmt ELSE stmt { $$ = opr(IF, 3, $3, $5, $7); }
| IF expr THEN stmt %prec IFX { $$ = opr(IF, 2, $2, $4); } //remove () and add THEN and correct the semantic rule
| IF expr THEN stmt ELSE stmt { $$ = opr(IF, 3, $2, $4, $6); } //remove () and add THEN correct the semantic rule
| '{' stmt_list '}' { $$ = $2; }
;
```

Check for calc3.h

- No changes should be necessary for **calc3.h** as it defines the data structures used by yacc parser and doesn't directly reference the syntax.

Check for calc3a.c (Interpreter)

- No changes are necessary unless the interpreter has specific assumptions about the syntax of **if** statements. The current structure for interpreting **IF** and **WHILE** statements should already be compatible with the changes that I've made to the yacc rules.

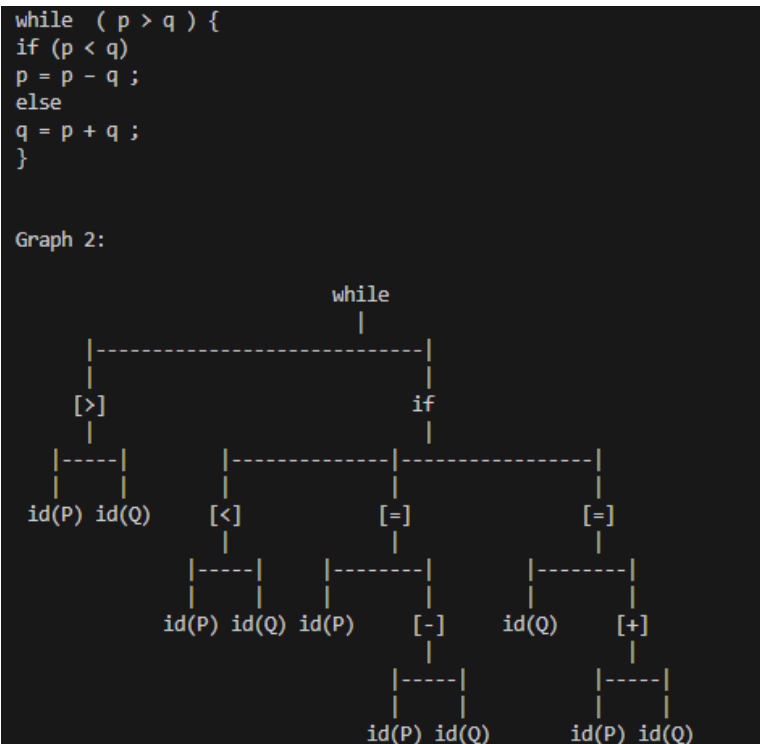
Check for calc3b.c (Compiler)

- Ensure that the compilation logic correctly handles the new syntax for **if-then-else** statements. The existing handling of **IF** should be adaptable to the new syntax without changes. The **case** block for **IF** in **calc3b.c** should already manage both 2 and 3 operand cases (i.e., if without else, and if with else).

Check for calc3g.c (Graph)

- This file generates the graphical representation of the syntax tree and should handle the new **IF** syntax without changes as long as it treats the **IF** node uniformly.

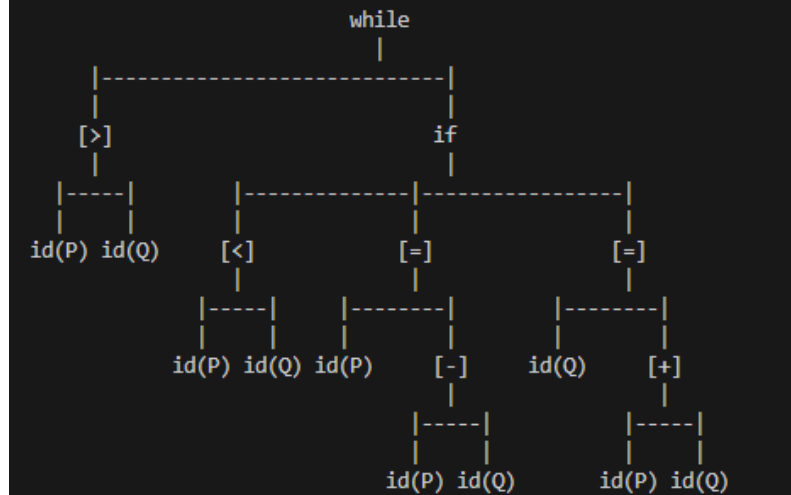
Before Change–



After Change-

```
while ( p > q ) {  
  if p < q then  
    p = p - q ;  
  else  
    q = p + q ;  
}
```

Graph 2:



Question 2

Your task is to design and implement a lexical analyzer for a programming language whose specifications are given below. The scanner identifies and outputs tokens (valid words and punctuation) in the source program. Its output is a token that can thereafter be used by the syntax analyzer to verify that the program is syntactically valid. When called, the lexical analyzer should extract the next token from the source program. The syntax of the language will be specified in Syntax phase.

Punctuation	Operators		Reserved words		
(;	+	<	else	integer	self
) ,	-	>	float	let	struct
{ .	*	<=	func	private	then
} :	/	>=	if	public	var
[->	=	or	impl	read	void
]	==	and	inherits	return	while
	<>	not			write

Lexical elements	
id	→ letter alphanum*
letter	→ a ... z A ... Z
alphanum	→ letter digit _
digit	→ 0 ... 9
integer	→ nonzero digit* 0
nonzero	→ 1 ... 9
float	→ integer fraction [e [+ -] integer]
fraction	→ .digit* nonzero .0

Comments:

- Inline comments start with // and end with the end of the line they appear in.
- Block comments start with /* and end with */ and may span over multiple lines.

Answer to the following tasks:

- Identify the lexical specification, expressed as regular expressions, that you used in the design of the lexical analyzer.
(**Note:** You should completely describe the data structure of your design and any changes that you may have applied to the original lexical specifications should be also included.)
- Describe the complete operation of your lexical analyzer using finite state diagram.
- Write Lex (and Yacc) files to construct a lexical analyzer that recognizes the above-stated tokens. It should be a function that returns a token data structure containing the information about the next token identified in the source program file. The token data structure should contains: (1) the token type; (2) the lexeme; and (3) the location, of the token in the source code.
(**Note:** The parser that Yacc generates should be able to call the lexical analyzer generated by Lex and perform parsing of the code generated by the given grammar. Your Lex file should be able to remove any white space from the code. All correspondences should be stored in a symbol table and printed out as output.)
- Identify all the possible lexical errors that the lexical analyzer might encounter and the possible error recovery techniques you could use. Choose one of them and discuss why you have chosen it.
- Include many test cases that test a wide variety of valid and invalid cases.

Question 2

(a) Identify the lexical specification

- For the Punctuations, Operators and Reserved words I defined token name as below. As usually token take in capitalized words.

Lexeme	Token Name
(	Punctuator_leftParan
)	Punctuator_rightParan
{	Punctuator_leftCurl
}	Punctuator_rightCurl
[	Punctuator_leftSqBr
]	Punctuator_rightSqBr
;	Punctuator_semiColon
,	Punctuator_comma
.	Punctuator_dot
:	Punctuator_colon
->	Punctuator_implies
+	Operaor_Add
-	OPERATOR_OF_Sub
*	OPERATOR_OF_Mul
/	OPERATOR_OF_Div
=	OPERATOR_OF_Assignment
==	OPERATOR_OF_Equal
<>	OPERATOR_OF_notEqual
<	OPERATOR_OF_lessThan
>	OPERATOR_OF_greaterThan
<=	OPERATOR_OF_lessThanOrEqualTo
>=	OPERATOR_OF_greaterThanOrEqualTo
or	OPERATOR_OF_OR
and	OPERATOR_OF_AND
not	OPERATOR_OF_NOT

else	RESERVED_WORD_OF_ELSE
float	RESERVED_WORD_OF_FLOAT
func	RESERVED_WORD_OF_FUNCTION
if	RESERVED_WORD_OF_IF
impl	RESERVED_WORD_OF_IMPLIES
inherits	RESERVED_WORD_OF_INHERITS
integer	RESERVED_WORD_OF_INTEGER
let	RESERVED_WORD_OF_LEQ
private	RESERVED_WORD_OF_PRIVATE
public	RESERVED_WORD_OF_PUBLIC
read	RESERVED_WORD_OF_READ
return	RESERVED_WORD_OF_RETURN
self	RESERVED_WORD_OF_SELF
struct	RESERVED_WORD_OF_STRUCT
then	RESERVED_WORD_OF_THEN
var	RESERVED_WORD_OF_VAR
void	RESERVED_WORD_OF_VOID
while	RESERVED_WORD_OF_WHILE

- **Lexical Elements and Regular Expressions**

There are 8 productions that need to be defined as Regular Expression and below I showcase for each how I defined.

1. **letter** $\rightarrow$ **a . . . z | A . . . Z** as **LETTER** [a-zA-Z]

- **Original Specification** - a . . . z | A . . . Z
- **Token name** - LETTER
- **Regular Expression** - [a-zA-Z]
- **Explanation** - A **letter** can be any lowercase or uppercase English alphabet character. This regular expression captures both ranges in a single expression.

2. **digit** → 0 . . . 9 as **DIGIT** [0-9]

- **Original Specification** - 0 . . . 9
- **Token Name** - DIGIT
- **Regular Expression** - [0-9]
- **Explanation** - A **digit** can be any digit from **0 to 9**. This is a straightforward character range in the regular expression.

3. **nonzero** → 1 . . . 9 as **NONZERO** [1-9]

- **Original Specification** - 1 . . . 9
- **Token Name** - NONZERO
- **Regular Expression** - [1-9]
- **Explanation:** A **nonzero** digit is any digit from 1 to 9.

4. **alphanum** → letter | digit | _ as **ALPHANUM** ({**LETTER**}{**DIGIT**}|_)

- **Original Specification** - letter | digit | _
- **Token Name** - ALPHANUM
- **Regular Expression** - ({**LETTER**}{**DIGIT**}|_)
- **Explanation** –
 - An **alphanum** can be a letter, a digit, or an underscore ‘_’. The regular expression combines the previously defined **LETTER** and **DIGIT** with an underscore.
 - **Using {} as Placeholders.**

In regular expressions, {} are often used as placeholders for other defined patterns. This allows for the reuse of patterns without rewriting them each time, making the expressions more modular and easier to understand.

- **Using | as the OR Operator**

The | character in regular expressions acts as an OR operator. It matches the pattern on the left or the pattern on the right. When grouped with parentheses, it can create choices between different sub-patterns.

5. **id** → letter alphanum* as **ID** {**LETTER**}{**ALPHANUM**}*

- **Original Specification** - letter alphanum*
- **Token Name** - ID
- **Regular Expression** - {LETTER}{ALPHANUM}*
- **Explanation** - An **id** starts with a **LETTER** and is followed by zero or more **ALPHANUM** characters. This captures typical identifier patterns.

6. **integer** $\rightarrow$ nonzero digit* | 0 as **INTEGER** (0 | {NONZERO}{DIGIT}*)

- **Original Specification** - nonzero digit* | 0
- **Token Name** - INTEGER
- **Regular Expression** - (0|{NONZERO}{DIGIT}*)
- **Explanation** - An **integer** can be 0 or a **NONZERO** digit followed by zero or more **DIGIT** characters. This covers both single-digit and multi-digit integers starting from 1-9, and the special case of 0.

7. **fraction** $\rightarrow$. digit* nonzero | .0 as **FRACTION** (\.{DIGIT}*{NONZERO}|\.0)

- **Original Specification** - .digit* nonzero | .0
- **Token Name** - FRACTION
- **Regular Expression** - (\.{DIGIT}*{NONZERO}|\.0)
- **Explanation:** A **fraction** can be a dot . followed by zero or more **DIGIT** characters ending with a **NONZERO** digit, or simply .0. This captures the decimal part of a float.

8. **EXPONENT** ([eE][+-]?{INTEGER})

Above is I newly defined Regular Expression that can be easy to show case in the float Regular expression.

- **Original Specification** - [e [+ | -] integer]
- **Token Name** - EXPONENT
- **Regular Expression** - ([eE][+-]?{INTEGER})
- **Explanation** - An **EXPONENT** part consists of e or E, optionally followed by + or -, and an **INTEGER**. This is a common pattern for scientific notation.

9. **float** $\rightarrow$ integer fraction [e [+ | -] integer] as

FLOAT ({INTEGER}{FRACTION}{EXPONENT}?|{INTEGER}{EXPONENT})|{INTEGER}{FRACTION})

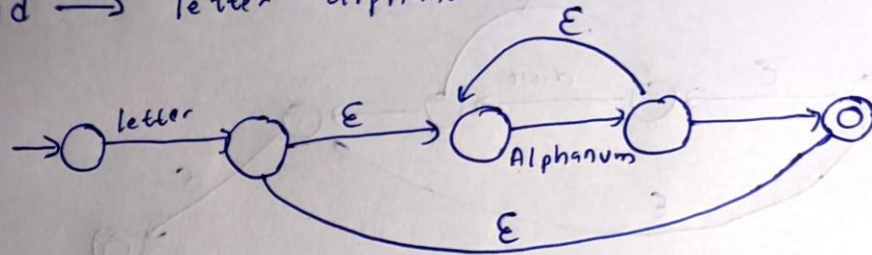
- **Original Specification:** integer fraction [e [+ | -] integer]
- **Regular Expression -**

(**{INTEGER}{FRACTION}{EXPONENT}?**|**{INTEGER}{EXPONENT}**){**INTEGER}{FRACTION}**)
- **Explanation -** A float can be,
 - An **INTEGER** followed by a **FRACTION** and an optional **EXPONENT**.
 - An **INTEGER** followed by an **EXPONENT**.
 - An **INTEGER** followed by a **FRACTION**.

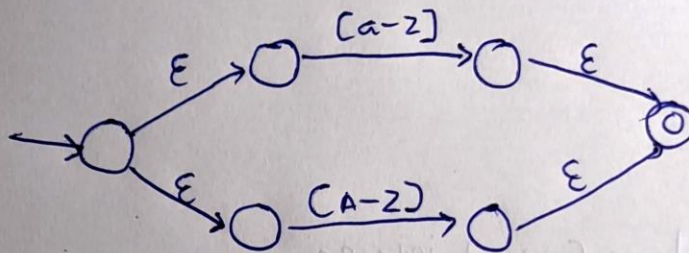
This definition captures different valid forms of floating-point numbers.

(b) Finite State Diagram

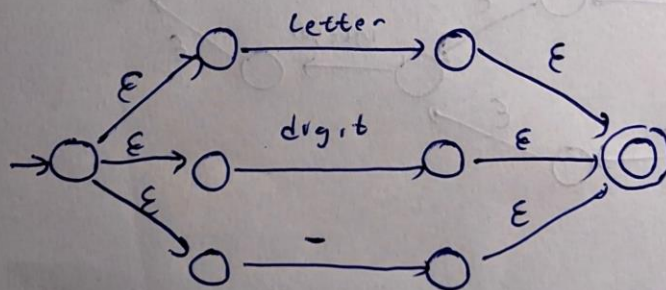
* $id \rightarrow letter\ alphanumeric^*$



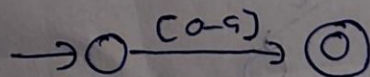
* $letter \rightarrow [a-z] \mid [A-Z]$



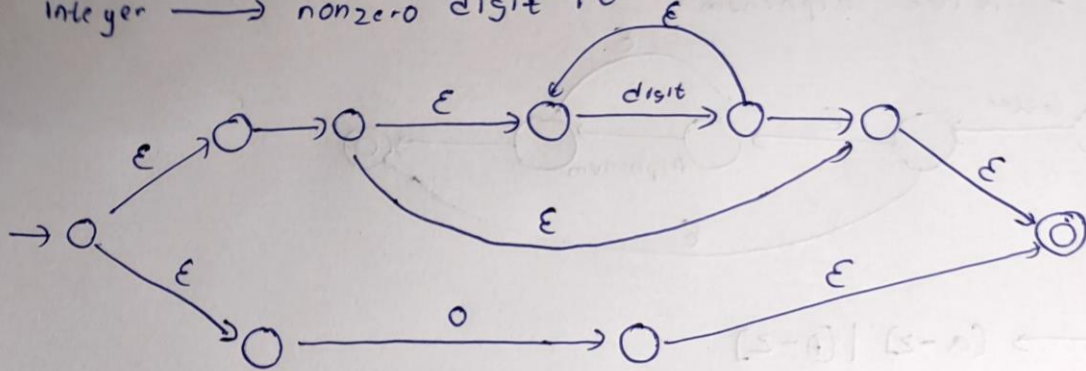
* $alphanumeric \rightarrow letter \mid digit \mid -$



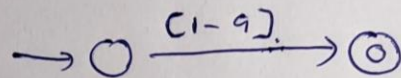
* $digit \rightarrow [0-9]$



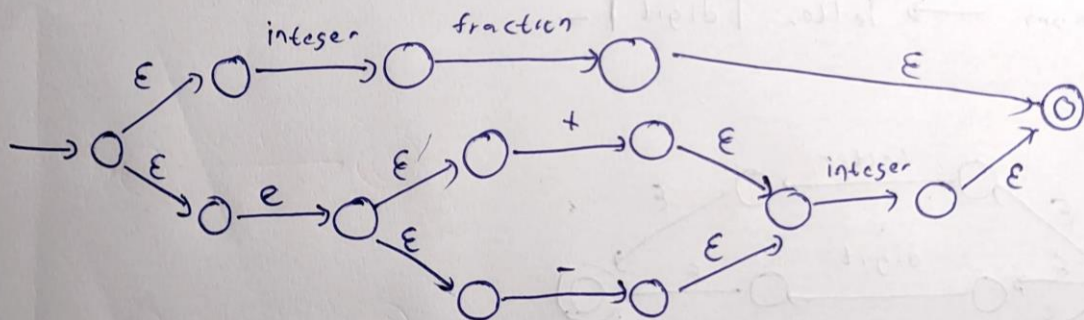
integer $\rightarrow$ nonzero digit* | 0



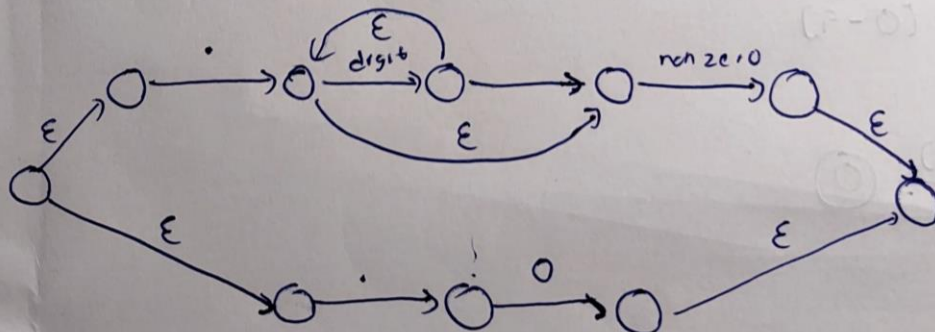
nonzero $\rightarrow$ [1-9]

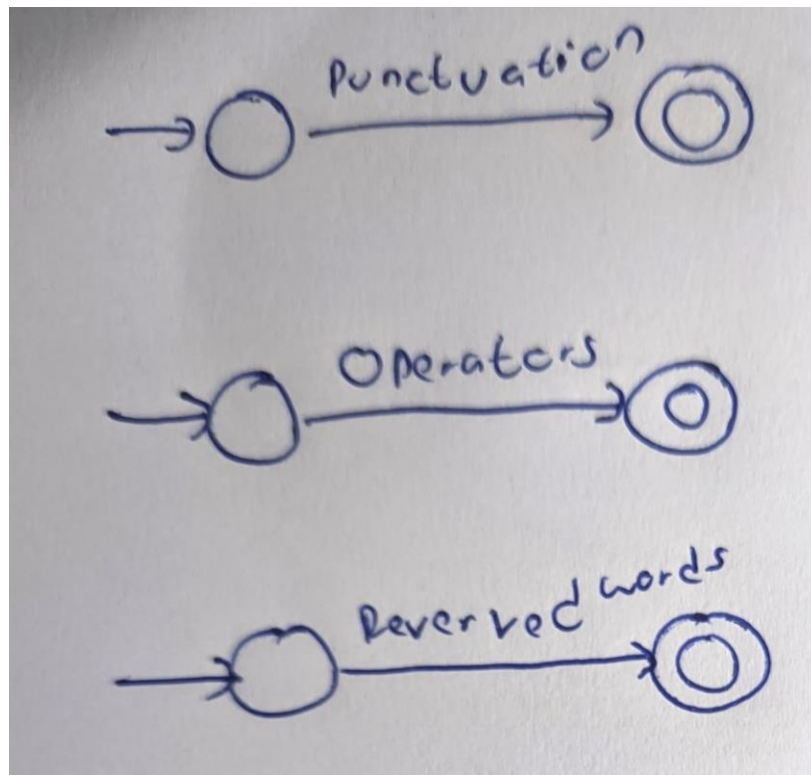


float $\rightarrow$ integer fraction | e[+|-] integer



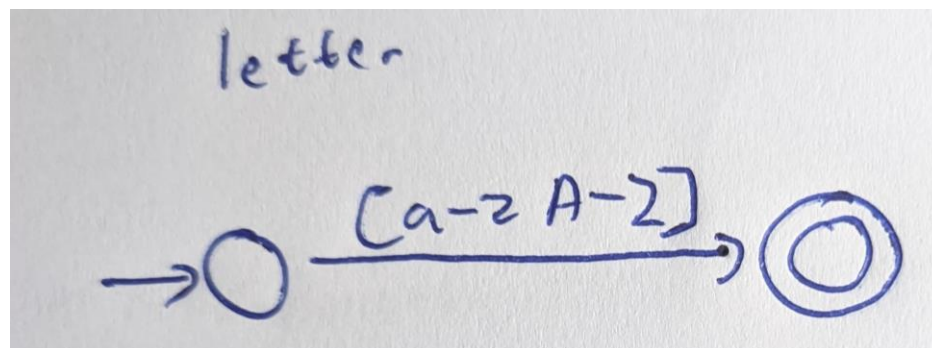
fraction $\rightarrow$. digit* non zero | . 0





- From the above given original productions of letter is taken as one single regular expression like below.

LETTER [a-zA-Z]

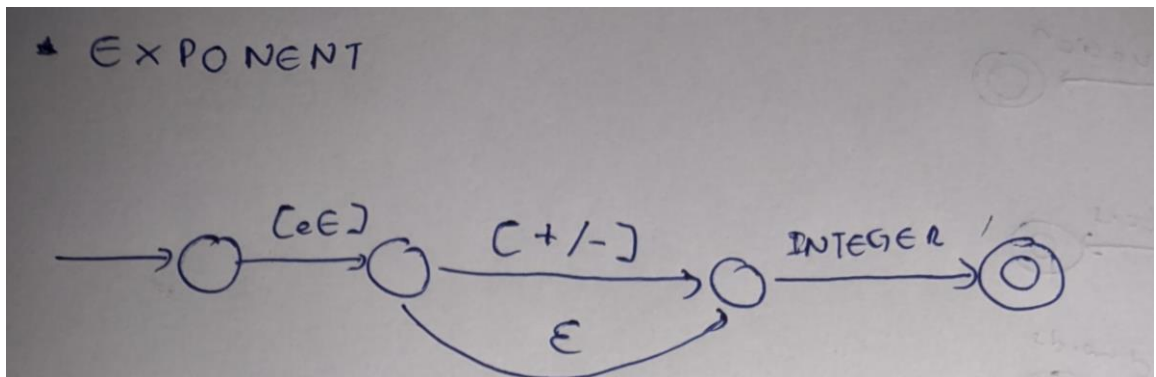


- For the below same as above finite state diagram procedure and different is only the token name as capitalize.

```
DIGIT [0-9]
NONZERO [1-9]
ALPHANUM ({LETTER}|{DIGIT}|_)
ID {LETTER}{ALPHANUM}*
INTEGER (0|{NONZERO}{DIGIT}*)
FRACTION (\.{DIGIT}*{NONZERO}|\.0)
```

- I create new regular expression as EXPONENT for that below is the finite state diagram and I used ? as zero or one occurrence.

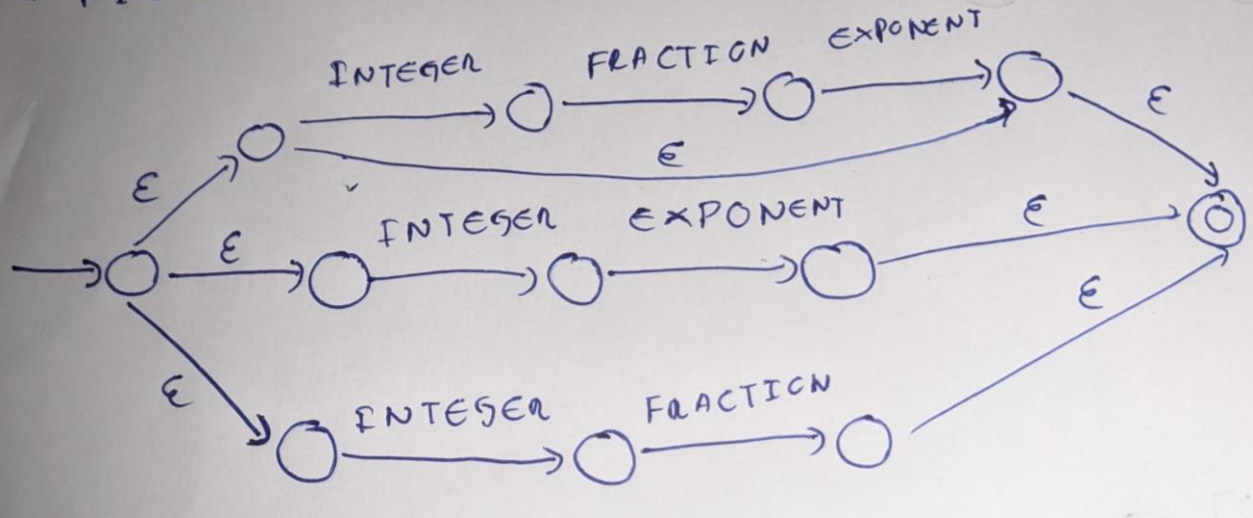
```
EXPONENT ([eE][+|-]?{INTEGER})
```



- For the float I create new Regular Expression but overall its gives the same above float meaning.

```
FLOAT ({INTEGER}{FRACTION}{EXPONENT}?|{INTEGER}{EXPONENT}|{INTEGER}{FRACTION})
```

• FLOAT



(c) Lexical analyzer

Part 1 - Using Flex and verified token is correct

Below is the Lexical Analyzer Code verification. I create verifying the token, lexeme and location.

```
%{
#include <stdio.h>
int total = 0;
int location = 1;
%}

LETTER [a-zA-Z]
DIGIT [0-9]
NONZERO [1-9]
ALPHANUM ({LETTER}|{DIGIT}|_)
ID {LETTER}{ALPHANUM}*
INTEGER (0|{NONZERO}{DIGIT}*)
FRACTION (\.{DIGIT}*{NONZERO}|\.\0)
EXPONENT ([eE][+|-]?{INTEGER})
FLOAT ({INTEGER}{FRACTION}{EXPONENT}?|{INTEGER}{EXPONENT}|{INTEGER}{FRACTION})
INVALID_FLOAT {DIGIT}+\.[0-9]*\.{DIGIT}*

%%

\n          {location++;}

"("          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_leftParan",
location);}
")"          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_rightParan",
location);}
"{"          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_leftCurl",
location);}
"}"          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_rightCurl",
location);}
"["          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_leftSqBr",
location);}
"]"          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_rightSqBr",
location);}
";"          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_semiColon",
location);}
","          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_comma",
location);}
"."          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_dot", location);}
":"          {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_colon",
location);}
"->"        {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Punctuator_implies",
location);}
```

```

"+"      {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_Add", location);}
"-"      {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_Sub", location);}
"*"      {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_Mul", location);}
"/"      {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_Div", location);}
"="      {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_Assignment",
location);}
"=="     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_Equal", location);}
"<>"     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_notEqual",
location);}
"<"      {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_lessThan",
location);}
">"      {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_greaterThan",
location);}
"<="     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_lessThanOrEqual",
location);}
">="     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_greaterThanOrEqual",
location);}
"or"     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_OR", location);}
"and"    {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_AND", location);}
"not"    {total++; printf("%-15s\t%-30s\t%d\n", yytext, "Operator_NOT", location);}
"else"   {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_ELSE",
location);}
"float"  {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_FLOAT",
location);}
"func"   {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_FUNCTION",
location);}
"if"     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_IF", location);}
"impl"   {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_IMPLIES",
location);}
"inherits" {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_INHERITS",
location);}
"integer" {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_INTEGER",
location);}
"let"     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_LET",
location);}
"private" {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_PRIVATE",
location);}
"public"  {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_PUBLIC",
location);}
"read"    {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_READ",
location);}
"return"  {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_RETURN",
location);}
"self"    {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_SELF",
location);}
"struct"  {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_STRUCT",
location);}
"then"    {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_THEN",
location);}
"var"     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_VAR",
location);}

```

```

"void"      {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_VOID",
location);}
"while"     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ReservedWord_WHILE",
location);}

{ID}        {total++; printf("%-15s\t%-30s\t%d\n", yytext, "ID", location);}
{INTEGER}   {total++; printf("%-15s\t%-30s\t%d\n", yytext, "INTEGER_NUMBER", location);}
{FLOAT}     {total++; printf("%-15s\t%-30s\t%d\n", yytext, "FLOAT_NUMBER", location);}

[ \t\n]+    ; /* ignore whitespace */
"//[^\n]*"   ; /* ignore Inline comments */
"/*(.|\n)*?" ; /* ignore Block comments */
.           {fprintf(stderr, "Warning: Unrecognized character '%s'
at location %d\n", yytext, location);}
{NONZERO}+{LETTER}+ { fprintf(stderr, "Warning: Invalid identifier '%s' starting with a
digit at location %d\n", yytext, location); }
{INVALID_FLOAT} { fprintf(stderr, "Error: Invalid floating-point literal '%s' at location
%d\n", yytext, location); }

%%

int main()
{
    printf("%-15s\t%-30s\t%s\n", "Lexeme", "Token Type", "Line Number");
    printf("-----\t-----\t-----\n");

    yylex();

    printf("\nTotal Tokens = %d\n", total);

    return 0;
}

int yywrap()
{
    return 1;
}

```

To get the out put more neatly I used below aspects.

- %s is used for strings.
- %d is used for integers.
- %-15s and %-30s are used to specify the width for each field, ensuring that the output is aligned in columns.
- \t is used to separate columns with a tab space.

Part 2 – Using both flex and bison verification.

From the above I can recognize the defined token are totally run. After that I create flex and bison file that identifies token from the flex and pass it to the bison as token and recognize the same one. Below is the code.

nawamUdayanga.l code file -

```
%{
#include <stdio.h>
#include "y.tab.h" // Include the header file generated by Bison
#include <string.h> // Include for strdup
extern int location; // Declare location as extern
%}

LETTER [a-zA-Z]
DIGIT [0-9]
NONZERO [1-9]
ALPHANUM ({LETTER}|{DIGIT}|_)
ID {LETTER}{ALPHANUM}*
INTEGER (0|{NONZERO}{DIGIT}*)
FRACTION (\.{DIGIT}*{NONZERO}|\.0)
EXPONENT ([eE][+|-]?{INTEGER})
FLOAT ({INTEGER}{FRACTION}{EXPONENT}?|{INTEGER}{EXPONENT}|{INTEGER}{FRACTION})
INVALID_FLOAT {DIGIT}+\.[0-9]*\.{DIGIT}*

%%

\n      { location++; }

"("      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_LEFTPARAN", location); return
PUNCT_LEFTPARAN; }
")"      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_RIGHTPARAN", location); return
PUNCT_RIGHTPARAN; }
"{"      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_LEFTCURL", location); return
PUNCT_LEFTCURL; }
"}"      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_RIGHTCURL", location); return
PUNCT_RIGHTCURL; }
"["      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_LEFTSQBR", location); return
PUNCT_LEFTSQBR; }
"]"      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_RIGHTSQBR", location); return
PUNCT_RIGHTSQBR; }
";"      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_SEMICOLON", location); return
PUNCT_SEMICOLON; }
","      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_COMMA", location); return
PUNCT_COMMA; }
"."      { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_DOT", location); return
PUNCT_DOT; }
```

```

":"          { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_COLON", location); return
PUNCT_COLON; }
"->"        { printf("%-15s\t%-20s\t%d\n", yytext, "PUNCT_IMPLIES", location); return
PUNCT_IMPLIES; }
"+"         { printf("%-15s\t%-20s\t%d\n", yytext, "OP_ADD", location); return OP_ADD; }
"_"         { printf("%-15s\t%-20s\t%d\n", yytext, "OP_SUB", location); return OP_SUB; }
"*"         { printf("%-15s\t%-20s\t%d\n", yytext, "OP_MUL", location); return OP_MUL; }
"/"         { printf("%-15s\t%-20s\t%d\n", yytext, "OP_DIV", location); return OP_DIV; }
"="         { printf("%-15s\t%-20s\t%d\n", yytext, "OP_ASSIGNMENT", location); return
OP_ASSIGNMENT; }
"=="        { printf("%-15s\t%-20s\t%d\n", yytext, "OP_EQUAL", location); return
OP_EQUAL; }
"<>"        { printf("%-15s\t%-20s\t%d\n", yytext, "OP_NOTEQUAL", location); return
OP_NOTEQUAL; }
"<"         { printf("%-15s\t%-20s\t%d\n", yytext, "OP_LESSTHAN", location); return
OP_LESSTHAN; }
">"         { printf("%-15s\t%-20s\t%d\n", yytext, "OP_GREATERTHAN", location); return
OP_GREATERTHAN; }
"<="        { printf("%-15s\t%-20s\t%d\n", yytext, "OP_LESSTHANOREQUAL", location);
return OP_LESSTHANOREQUAL; }
">="        { printf("%-15s\t%-20s\t%d\n", yytext, "OP_GREATERTHANOREQUAL", location);
return OP_GREATERTHANOREQUAL; }
"or"        { printf("%-15s\t%-20s\t%d\n", yytext, "OP_OR", location); return OP_OR; }
"and"       { printf("%-15s\t%-20s\t%d\n", yytext, "OP_AND", location); return OP_AND; }
"not"       { printf("%-15s\t%-20s\t%d\n", yytext, "OP_NOT", location); return OP_NOT; }
"else"      { printf("%-15s\t%-20s\t%d\n", yytext, "RES_ELSE", location); return
RES_ELSE; }
"float"     { printf("%-15s\t%-20s\t%d\n", yytext, "RES_FLOAT", location); return
RES_FLOAT; }
"func"      { printf("%-15s\t%-20s\t%d\n", yytext, "RES_FUNC", location); return
RES_FUNC; }
"if"        { printf("%-15s\t%-20s\t%d\n", yytext, "RES_IF", location); return RES_IF; }
"impl"      { printf("%-15s\t%-20s\t%d\n", yytext, "RES_IMPL", location); return
RES_IMPL; }
"inherits"  { printf("%-15s\t%-20s\t%d\n", yytext, "RES_INHERITS", location); return
RES_INHERITS; }
"integer"   { printf("%-15s\t%-20s\t%d\n", yytext, "RES_INTEGER", location); return
RES_INTEGER; }
"let"       { printf("%-15s\t%-20s\t%d\n", yytext, "RES_LET", location); return RES_LET;
}
"private"   { printf("%-15s\t%-20s\t%d\n", yytext, "RES_PRIVATE", location); return
RES_PRIVATE; }
"public"    { printf("%-15s\t%-20s\t%d\n", yytext, "RES_PUBLIC", location); return
RES_PUBLIC; }
"read"      { printf("%-15s\t%-20s\t%d\n", yytext, "RES_READ", location); return
RES_READ; }
"return"    { printf("%-15s\t%-20s\t%d\n", yytext, "RES_RETURN", location); return
RES_RETURN; }
"self"      { printf("%-15s\t%-20s\t%d\n", yytext, "RES_SELF", location); return
RES_SELF; }

```

```

"struct"      { printf("%-15s\t%-20s\t%d\n", yytext, "RES_STRUCT", location); return
RES_STRUCT; }
"then"        { printf("%-15s\t%-20s\t%d\n", yytext, "RES_THEN", location); return
RES_THEN; }
"var"         { printf("%-15s\t%-20s\t%d\n", yytext, "RES_VAR", location); return RES_VAR;
}
"void"        { printf("%-15s\t%-20s\t%d\n", yytext, "RES_VOID", location); return
RES_VOID; }
"while"       { printf("%-15s\t%-20s\t%d\n", yytext, "RES_WHILE", location); return
RES_WHILE; }

{ID}          { printf("%-15s\t%-20s\t%d\n", yytext, "IDENTIFIER", location); yylval.str =
strdup(yytext); return IDENTIFIER; }
{INTEGER}     { printf("%-15s\t%-20s\t%d\n", yytext, "INTEGER_NUMBER", location); yylval.num =
atoi(yytext); return INTEGER_NUMBER; }
{FLOAT}       { printf("%-15s\t%-20s\t%d\n", yytext, "FLOAT_NUMBER", location); yylval.fnum =
atof(yytext); return FLOAT_NUMBER; }

[ \t\n]+      ; /* ignore whitespace */
"//[^\n]*"     ; /* ignore inline comments */
"/*(.[\n])*?*/" ; /* ignore block comments */

. { fprintf(stderr, "Warning: Unrecognized character '%s' at location %d\n", yytext,
location); }
{NONZERO}+{LETTER}+ { fprintf(stderr, "Warning: Invalid identifier '%s' starting with a
digit at location %d\n", yytext, location); }
{INVALID_FLOAT} { fprintf(stderr, "Error: Invalid floating-point literal '%s' at location
%d\n", yytext, location); }
"/*"([^\n]|(\n+[^/]))*$ { fprintf(stderr, "Error: Unclosed block comment at location %d\n",
location); }

%%

int yywrap() {
    return 1;
}

```

nawamUdayanga.y code file -

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int yylex();
extern int yyparse();
extern FILE *yyin;
void yyerror(const char *s);

int location = 1; // Define location here

%}

%union {
    char *str;
    int num;
    float fnum;
}

%token <str> IDENTIFIER
%token <num> INTEGER_NUMBER
%token <fnum> FLOAT_NUMBER
%token PUNCT_LEFTPARAN PUNCT_RIGHTPARAN PUNCT_LEFTCURL PUNCT_RIGHTCURL PUNCT_LEFTSQBR
PUNCT_RIGHTSQBR PUNCT_SEMICOLON PUNCT_COMMA PUNCT_DOT PUNCT_COLON PUNCT_IMPLIES
%token OP_ADD OP_SUB OP_MUL OP_DIV OP_ASSIGNMENT OP_EQUAL OP_NOTEQUAL OP_LESSTHAN
OP_GREATERTHAN OP_LESSTHANOREQUAL OP_GREATERTHANOREQUAL OP_OR OP_AND OP_NOT
%token RES_ELSE RES_FLOAT RES_FUNC RES_IF RES_IMPL RES_INHERITS RES_INTEGER RES_LET
RES_PRIVATE RES_PUBLIC RES_READ RES_RETURN RES_SELF RES_STRUCT RES_THEN RES_VAR RES_VOID
RES_WHILE

%type <str> statement
%type <str> line

%%

input:
    /* empty */
    | input line
    ;

line:
    statement '\n' { printf("Lexeme: %-15s Token Type: %-30s Line Number: %d\n", $1,
$1, location); }
    | error '\n' { yyerrok; }
    ;
```

statement:

```
IDENTIFIER { $$ = "IDENTIFIER"; }
| INTEGER_NUMBER { $$ = "INTEGER_NUMBER"; }
| FLOAT_NUMBER { $$ = "FLOAT_NUMBER"; }
| PUNCT_LEFTPARAM { $$ = "PUNCT_LEFTPARAM"; }
| PUNCT_RIGHTPARAM { $$ = "PUNCT_RIGHTPARAM"; }
| PUNCT_LEFTCURL { $$ = "PUNCT_LEFTCURL"; }
| PUNCT_RIGHTCURL { $$ = "PUNCT_RIGHTCURL"; }
| PUNCT_LEFTSQBR { $$ = "PUNCT_LEFTSQBR"; }
| PUNCT_RIGHTSQBR { $$ = "PUNCT_RIGHTSQBR"; }
| PUNCT_SEMICOLON { $$ = "PUNCT_SEMICOLON"; }
| PUNCT_COMMA { $$ = "PUNCT_COMMA"; }
| PUNCT_DOT { $$ = "PUNCT_DOT"; }
| PUNCT_COLON { $$ = "PUNCT_COLON"; }
| PUNCT_IMPLIES { $$ = "PUNCT_IMPLIES"; }
| OP_ADD { $$ = "OP_ADD"; }
| OP_SUB { $$ = "OP_SUB"; }
| OP_MUL { $$ = "OP_MUL"; }
| OP_DIV { $$ = "OP_DIV"; }
| OP_ASSIGNMENT { $$ = "OP_ASSIGNMENT"; }
| OP_EQUAL { $$ = "OP_EQUAL"; }
| OP_NOTEQUAL { $$ = "OP_NOTEQUAL"; }
| OP_LESSTHAN { $$ = "OP_LESSTHAN"; }
| OP_GREATERTHAN { $$ = "OP_GREATERTHAN"; }
| OP_LESSTHANOREQUAL { $$ = "OP_LESSTHANOREQUAL"; }
| OP_GREATERTHANOREQUAL { $$ = "OP_GREATERTHANOREQUAL"; }
| OP_OR { $$ = "OP_OR"; }
| OP_AND { $$ = "OP_AND"; }
| OP_NOT { $$ = "OP_NOT"; }
| RES_ELSE { $$ = "RES_ELSE"; }
| RES_FLOAT { $$ = "RES_FLOAT"; }
| RES_FUNC { $$ = "RES_FUNC"; }
| RES_IF { $$ = "RES_IF"; }
| RES_IMPL { $$ = "RES_IMPL"; }
| RES_INHERITS { $$ = "RES_INHERITS"; }
| RES_INTEGER { $$ = "RES_INTEGER"; }
| RES_LET { $$ = "RES_LET"; }
| RES_PRIVATE { $$ = "RES_PRIVATE"; }
| RES_PUBLIC { $$ = "RES_PUBLIC"; }
| RES_READ { $$ = "RES_READ"; }
| RES_RETURN { $$ = "RES_RETURN"; }
| RES_SELF { $$ = "RES_SELF"; }
| RES_STRUCT { $$ = "RES_STRUCT"; }
| RES_THEN { $$ = "RES_THEN"; }
| RES_VAR { $$ = "RES_VAR"; }
| RES_VOID { $$ = "RES_VOID"; }
| RES_WHILE { $$ = "RES_WHILE"; }
```

;

%%

```

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *file = fopen(argv[1], "r");
        if (!file) {
            fprintf(stderr, "Could not open %s\n", argv[1]);
            return 1;
        }
        yyin = file;
    }
    printf("Lexeme          Token Type          Line Number\n");
    printf("-----          -\n");
    yyparse();
    return 0;
}

```

(d) Lexical Error recognized.

Unrecognized Characters

- Characters that do not match any token pattern defined in the language.
Example: - @, #, etc.
- The rule '.' captures any character that does not match other rules and reports it as an unrecognized character and report the error and continue processing the next character.

Snippet code –

```

. {fprintf(stderr, "Warning: Unrecognized character '%s'
at location %d\n", yytext, location);}

```

Invalid Identifiers

- Sequences that do not conform to any valid token format.
- Example: - 123abc, 1a etc. (assuming identifiers cannot start with digits).
- Specific patterns like {NONZERO}+{LETTER}+ detect tokens that are invalid, such as identifiers starting with a digit. Then the show the error and continue processing the next token.

Snippet Code –

```
{NONZERO}+{LETTER}+ { fprintf(stderr, "Warning: Invalid identifier '%s' starting with a digit at location %d\n", yytext, location); }
```

Malformed Numbers

- Numbers that do not conform to the correct format for integers or floating-point numbers.
- Example: - **45.67.89**.
- **Handler:** The `INVALID_FLOAT` pattern detects floating-point numbers with multiple decimal points.
- **Recovery:** Report the error and continue processing the next token.

Snippet code –

```
%{  
.  
.  
.  
%}  
  
.  
.  
.  
.  
  
INVALID_FLOAT {DIGIT}+\. [0-9]*\.{DIGIT}*  
  
%%  
.  
.  
.  
  
{INVALID_FLOAT} { fprintf(stderr, "Error: Invalid floating-point literal '%s' at location %d\n", yytext, location); }  
%%
```

(e) Test Cases

Part 1 – flex only

Unrecognized Characters

```
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_flex_only>a.exe
Lexeme          Token Type          Line Number
-----
#
Warning: Unrecognized character '#' at location 1
@
Warning: Unrecognized character '@' at location 2
^Z

Total Tokens = 0

F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_flex_only>
```

Invalid Identifiers

```
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_flex_only>a.exe
Lexeme          Token Type          Line Number
-----
123a
Warning: Invalid identifier '123a' starting with a digit at location 1
1a
Warning: Invalid identifier '1a' starting with a digit at location 2
^Z

Total Tokens = 0

F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_flex_only>
```

Malformed Numbers

1. **Valid Floating-Point Literals** – example (float val1 = 123.456; , float val2 = 0.123; , float val3 = 123.0; , float val4 = 1.23e10; , float val5 = 1.23E-10;)

output -

```
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_flex_only>a.exe
Lexeme          Token Type          Line Number
-----
float val1 = 123.456;
float           ReservedWord_FLOAT          1
val1            ID                          1
=              Operator_Assignment          1
123.456         FLOAT_NUMBER                     1
;              Punctuator_semiColon         1
float val2 = 0.123;
float           ReservedWord_FLOAT          2
val2            ID                          2
=              Operator_Assignment          2
0.123           FLOAT_NUMBER                     2
;              Punctuator_semiColon         2
float val3 = 123.0;
float           ReservedWord_FLOAT          3
val3            ID                          3
=              Operator_Assignment          3
123.0           FLOAT_NUMBER                     3
;              Punctuator_semiColon         3
float val4 = 1.23e10;
float           ReservedWord_FLOAT          4
val4            ID                          4
=              Operator_Assignment          4
1.23e10         FLOAT_NUMBER                     4
;              Punctuator_semiColon         4
float val5 = 1.23E-10;
float           ReservedWord_FLOAT          5
val5            ID                          5
=              Operator_Assignment          5
1.23E-10        FLOAT_NUMBER                     5
;              Punctuator_semiColon         5
^Z

Total Tokens = 25
```

2. Invalid Floating-Point Literals (Malformed)

- float val6 = 123.456.789;
- float val7 = 1.23.45e10;
- float val8 = 123..456;

output –

```
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_flex_only>a.exe
Lexeme          Token Type          Line Number
-----
float val6 = 123.456.789;
float           ReservedWord_FLOAT          1
val6            ID                          1
=              Operator_Assignment          1
Error: Invalid floating-point literal '123.456.789' at location 1
;              Punctuator_semiColon          1
float val7 = 1.23.45e10;
float           ReservedWord_FLOAT          1
val7            ID                          1
=              Operator_Assignment          1
Error: Invalid floating-point literal '1.23.45' at location 1
e10            ID                          1
;              Punctuator_semiColon          1
float val8 = 123..456;
float           ReservedWord_FLOAT          1
val8            ID                          1
=              Operator_Assignment          1
Error: Invalid floating-point literal '123..456' at location 1
;              Punctuator_semiColon          1
^Z

Total Tokens = 13

F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_flex_only>
```

Part 2 – both flex and bison

```
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_with_flex_and_bison>bison -y -d nawamUdayanga.y
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_with_flex_and_bison>flex nawamUdayanga.l
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_with_flex_and_bison>gcc -c y.tab.c lex.yy.c
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_with_flex_and_bison>gcc y.tab.o lex.yy.o -o nawamUdayanga
F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_with_flex_and_bison>nawamUdayanga
Lexeme          Token Type      Line Number
-----
@
Warning: Unrecognized character '@' at location 1
#
Warning: Unrecognized character '#' at location 2
1a
Warning: Invalid identifier '1a' starting with a digit at location 3
123v
Warning: Invalid identifier '123v' starting with a digit at location 4
^Z

F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_with_flex_and_bison>nawamUdayanga
Lexeme          Token Type      Line Number
-----
3.3
3.3             FLOAT_NUMBER    1
3.3.3
Error: Invalid floating-point literal '3.3.3' at location 2
^Z
Error: syntax error

F:\NAWAM\LEXICAL ANALYSER\Question_2_Answer_with_flex_and_bison>
```